SOLMAE: Lightweight Post-Quantum Signature based on NTRU lattices with Hybrid Sampling

Kwangjo Kim

IRCS and KAIST(Korea), CSU(USA),
kkj@kaist.ac.kr

Abstract. The paper introduces SOLMAE, a lightweight post-quantum signature scheme that follows the traditional hash-and-sign paradigm of Gentry–Peikert–Vaikuntanathan and is instantiated over NTRU lattices using hybrid Gaussian samplers. As a natural successor to earlier designs including Falcon, Mitaka and Antrag, SOLMAE combines the strengths of these approaches. In particular, SOLMAE positions itself as offering a unified framework that achieves improved efficiency and security trade-offs over Falcon, Mitaka, and Antrag, continuing the evolution of efficient lattice-based signatures over structured lattices. SOLMAE leverages the simplicity, speed, and parallelizability of Mitaka while matching the high security and compact key and signature sizes of Falcon. This is achieved through a novel key-generation algorithm that enhances security and removes the rigidity present in Falcon. At the same time, it retains full parameter flexibility and a fast signing procedure. The design is further compatible with recent ellipsoidal Gaussian sampling techniques, enabling even smaller signatures. Altogether, SOLMAE, suitable for resource-constrained environment, establishes a new efficiency point in lattice-based signatures, with remaining implementation considerations deferred to the conclusion.

Keywords: Falcon · Lattice-based cryptography · Mitaka · Antrag · Hash-and-sign paradigm · NTRU lattices · Hybrid Sampling

1 Introduction

Designing cryptographically strong primitives—such as digital signatures and key encapsulation mechanisms—is a highly challenging task that cannot be achieved by a single expert within a short time frame. It requires the collective effort of a group of skilled designers who possess a deep understanding of all known attacks, from both theoretical and implementation perspectives, and who can anticipate plausible future attack vectors. Our design team for SOLMAE, which is an abbreviation of quantum–**S**ecure alg**O**rithm for **L**ong-term **M**essage **A**uthentication and **E**ncryption, https://solmae-sign.info, composed of leading cryptographers from

around the world, has initiated the development of long-term secure and post-quantum digital signature schemes based on NTRU lattices. These lattice structures have been extensively studied and rigorously scrutinized by the cryptographic community for over two decades, providing a solid foundation for building secure and practical post-quantum cryptographic solutions.

After six-years NIST PQC standardization project (2017–2022,) Falcon [41] was selected as one of the final quantum-safe digital signature algorithms for based on the NTRU trapdoor together with Dilithium and SPHINCS+ in 2022. Falcon is a hash-and-sign signature scheme based on paradigm on the Gentry, Peikert and Vaikuntanathan (GPV) [16] based on the lattice problem. To be efficiently instantiated, this framework needs a class of lattices with efficiently computational *trapdoor bases* for the signing procedure. Ducas, Lyubashevsky and Prest (DLP) [8] showed that NTRU lattices are a nice class of lattices. Thus, short Gaussian vectors are sampled in a public NTRU lattice; a quasi-linear (time and memory) but complex procedure called Fast Fourier Sampling was described by Ducas and Prest [9]. Falcon achieves the most compact signatures, and the most compact verification key and signatures combined sizes, and boasts verification times on par with elliptic curve signatures.

Inspired by Falcon's design, Espitau *et al.* in 2023 suggested so–called Antrag [13] in order to improve Mitaka [10] without loss of security covering all NIST five levels of security using the degree of cyclotomic ring from 512 to 1024 over specific cyclotomic polynomials under the prime modulus but is not limited to the power of 2.

Leveraging the advantages of Falcon, Mitaka and Antrag, SOLMAE is yet another post–quantum signature based on NTRU trapdoor and achieves *better performance* for the *same security and advantages* as Falcon which focused only on NIST I and V levels of security. More precisely, SOLMAE offers a unified framework achieving improved efficiency and security trade-offs over Falcon, Mitaka and Antrag. At a high–level, it removes the inherent technicality of the sampling procedure, and most of its induced complexity from an implementation standpoint, for *free*, that is with no loss of efficiency. The simplicity of our design translates into faster operations while preserving signature and verification key sizes, in addition to allowing for additional features absent from Falcon, such as enjoying less expensive masking, and being parallelizable.

Figure 1 presents an overview of the overall concept of SOLMAE.

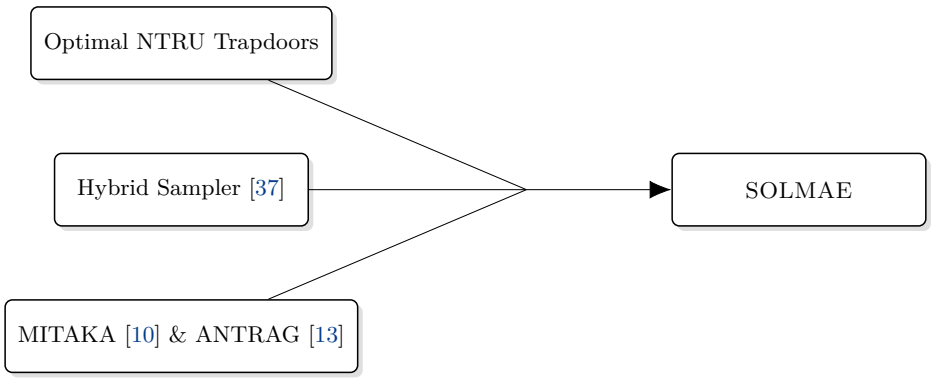


Figure 1: Overview of SOLMAE

While its predecessor Falcon could be summed up as "an efficient instantiation of the GPV framework", SOLMAE takes it one step further. The ingredients behind the boxes in Figure 1 are as follows:

- An **optimally tuned key generation algorithm**, enhancing the security of our new sampler to that of Falcon's level I and V;
- The **hybrid sampler** is a faster, simpler, parallelizable and maskable Gaussian sampler to generate signatures;
- **Easy implementation** by assembling all the advantages of Mitaka and Antrag to make the scheme faster and simpler for practical deployment.

Our main contributions are:

- A new NTRU-based signature scheme eliminating rejection-based key generation.
- A hybrid Gaussian sampler achieving quasi-linear complexity.
- A masking-friendly design enabling t-NI and t-SNI secure implementations.
- Improved practical performance compared to Falcon in signing.

SOLMAE is not intended to replace Falcon in standardization, but rather to explore a new design point emphasizing simplicity, maskability, and implementation efficiency. The organization of this paper is as follows. Section 2 reviews the mathematical and algebraic preliminaries. Section 3 presents the SOLMAE construction, including the `KeyGen`, `Sign`, `Sample`, and `Verif` algorithms. Section 4 provides a concrete security analysis based on the core-SVP framework. Section 5 evaluates the performance of SOLMAE through implementation benchmarks and comparisons with

Falcon. Section 6 describes the encoding rules and Falcon-compatible data formats used in SOLMAE. Finally, Section 7 concludes the paper with remarks on practical implications and future directions.

2 Preliminaries

Vectors are denoted in bold lower case, and considered to be in column. Matrices are in bold upper case. When we say that a matrix is a basis of a space, we mean that the column vectors of the matrix are the basis. The ℓ_2-norm of a vector $\mathbf{x} = (x_1, \ldots, x_d)$ is $\|\mathbf{x}\| = (\sum_i |x_i|^2)^{1/2}$ and its ℓ_∞-norm is $\|\mathbf{x}\|_\infty = \max_i |x_i|$.

Lattices A lattice is a discrete subgroup of $\mathbb{R}^n$. Equivalently, it is the set of *integer* linear combinations obtained from a basis $\mathbf{B}$ of $\mathbb{R}^n$. The volume of a lattice is $\det \mathbf{B}$ for any of its basis.

Cyclotomic powers-of-two rings For $d = 2^n$, we let $K = \mathbb{Q}[X]/(X^d+1)$ be the d-th cyclotomic field. We often work in the subring $R = \mathbb{Z}[X]/(X^d+1)$, and sometimes in the overring $K_\mathbb{R} = \mathbb{R}[X]/(X^d+1)$. Let $\zeta_j = \exp(i(2j-1)\pi/d)$ for $1 \leq j \leq d$ be the d-th primitive root of 1, and for all $f \in K_\mathbb{R}$, let $\varphi_i(f) = f(\zeta_i)$. A polynomial in $K_\mathbb{R}$ can be represented in several ways. The first is the so-called coefficient embedding $f = \sum f_i X^i \mapsto \mathbf{f} = (f_0, \ldots, f_{d-1})$. Another is the canonical embedding $f \mapsto \varphi(f) = (\varphi_1(f), \ldots, \varphi_d(f))$. The map φ is also known as the Discrete Fourier Transform (DFT) and in particular, φ maps the polynomial multiplication in $K_\mathbb{R}$ to a coordinate-wise multiplication. The set of f's such that all $\varphi_i(f) \in \mathbb{R}_+^*$ is denoted by $K_\mathbb{R}^{++}$. We have $\|\varphi(f)\| = \sqrt{d}\|\mathbf{f}\|$. We let f^* be the complex conjugate of f and

$$f^* = f_0 - f_{d-1}X - \ldots - f_1 X^{d-1}, \quad \varphi(f^*) = (\overline{\varphi_i(f)})_i.$$

The third representation of cyclotomic elements is by matrices of multiplication:

$$f \mapsto [f] := \begin{bmatrix} f_0 & -f_{d-1} & \ldots & -f_1 \\ f_1 & f_0 & \ldots & -f_2 \\ \vdots & & \ddots & \vdots \\ f_{d-1} & f_{d-2} & \ldots & f_0 \end{bmatrix}.$$

We have $[f^*] = [f]^t$.

In algorithms, we write $(f_i)_i \in \mathbb{R}^d$ or $f \in K_\mathbb{R}$: the former highlights that the vector of coefficient is considered, while the latter implies that the Fourier representation is used.

Algebraic Gram-Schmidt For $(f,g),(F,G) \in K_{\mathbb{R}}^{2\times 2}$, we define $\langle (f,g),(F,G)\rangle_K = f^*F + g^*G$. The Gram-Schmidt orthogonalization of $(F,G) \in K_{\mathbb{R}}^2$ with respect to (f,g) is

$$(\widetilde{F},\widetilde{G}) = (F,G) - \frac{\langle (f,g),(F,G)\rangle_K}{\langle (f,g),(f,g)\rangle_K} \cdot (f,g),$$

and one checks that $\langle (f,g),(\widetilde{F},\widetilde{G})\rangle_K = 0$.

NTRU lattices Let q be an integer, and $f \in R$ such that f is invertible modulo q (equivalently, $\det[f]$ is coprime to q). Let $h = g/f \bmod q$ and consider the NTRU module associated with h:

$$\mathcal{M}_{\mathrm{NTRU}} = \{(u,v) \in R^2 : hu - v = 0 \bmod q\},$$

and its lattice version

$$\mathcal{L}_{\mathrm{NTRU}} = \{(\mathbf{u},\mathbf{v}) \in \mathbb{Z}^{2d} : [h]\mathbf{u} - \mathbf{v} = 0 \bmod q\}.$$

This lattice has a volume q^d. Over R, it is generated by (f,g) and any (F,G) such that $fG - gF = q$. For such a pair $(f,g),(F,G)$, this means that $\mathcal{L}_{\mathrm{NTRU}}$ has a basis of the form

$$\mathbf{B}_{f,g} = \begin{bmatrix} [f] & [F] \\ [g] & [G] \end{bmatrix}.$$

One checks that $([h], -\mathrm{Id}_d) \cdot \mathbf{B}_{f,g} = 0 \bmod q$, so the verification key is h. More details are given in Section 3.2. The NTRU-search problem is as follows: given $h = g/f \mod q$, find any $(f' = x^i f, g' = x^i g)$. In its decision variant, one must distinguish $h = g/f \bmod q$ from a uniformly random $h \in R_q := \mathbb{Z}[X]/(q, X^d+1) = (\mathbb{Z}/q\mathbb{Z})[X]/(X^d+1)$. These problems are assumed to be intractable for large d.

Quality of an NTRU basis The secret basis will not be any pair, as it also needs to enable the sampling of short Gaussian vectors in $\mathcal{L}_{\mathrm{NTRU}}$ by hybrid sampling. The *quality* of an NTRU basis $\mathbf{B}_{f,g}$ quantifies this:

$$\mathcal{Q}(f,g) = \max_{1\le i\le d/2} \max\left(\frac{|\varphi_i(f)|^2 + |\varphi_i(g)|^2}{q}, \frac{q}{|\varphi_i(f)|^2 + |\varphi_i(g)|^2}\right)^{1/2}.$$

Note that only half of the embeddings are needed, since the remaining ones correspond to the complex conjugates. It is one of the main parameters driving the security of the scheme. It cannot be lower than 1, but the closer it is to 1, the harder forgery attacks are.

Gaussian distributions The Gaussian function centered at $\mathbf{c} \in \mathbb{R}^d$ and (positive definite) covariance Σ is $\rho_{\mathbf{c},\Sigma}(\mathbf{x}) = \exp(-\frac{1}{2}(\mathbf{x}-\mathbf{c})^t \Sigma^{-1}(\mathbf{x}-\mathbf{c}))$. The normal distribution $\mathcal{N}_{\mathbf{c},\Sigma}$ centered at $\mathbf{c}$ and covariance Σ has density proportional to $\rho_{\mathbf{c},\Sigma}$. When we write $x \leftarrow \mathcal{N}_{\Sigma}^{K_{\mathbb{R}}}$, we mean that the corresponding d dimensional vector $\sqrt{\frac{2}{d}}(\Re\varphi_1(x), \Im\varphi_1(x) \ldots, \Re\varphi_{d/2}(x), \Im\varphi_{d/2}(x))$ has a distribution $\mathcal{N}_{\Sigma}$, where $\Re z$ and $\Im z$ are the real and imaginary parts of complex z, respectively. For a lattice $\mathcal{L} \subset \mathbb{R}^d$, the discrete Gaussian distribution over $\mathcal{L}$ with parameters $\mathbf{c} \in \mathbb{R}^d$ and Σ is defined for all $\mathbf{x} \in \mathcal{L}$ as

$$D_{\mathcal{L},\mathbf{c},\Sigma}(\mathbf{x}) = \frac{\rho_{\mathbf{c},\Sigma}(\mathbf{x})}{\rho_{\Sigma}(\mathcal{L}-\mathbf{c})}.$$

We omit the center if it is 0. When Σ is a scalar matrix $s^2\mathbf{I}$, we note $\mathcal{N}_s$ or $D_{\mathcal{L},\mathbf{t},s}$.

3 Specifications

We first detail the principles of the GPV framework. This highlights the necessary ingredients for an efficient signature scheme. The next sections are devoted to the scheme's description, and how we instantiate the triple `KeyGen`, `Sign`, `Verif`. The concrete values for the many parameters to be introduced can be found at the end of the section in Table 1.

3.1 High-level view of the SOLMAE's signature scheme

As in any signature scheme, we need to instantiate three algorithms `KeyGen`, `Sign` and `Verif`. Moreover, the GPV framework has the following requirements:

- a class of lattices where `KeyGen` computes trapdoor pairs;
- `Sign` uses a `Sample` procedure to generate random vectors in these lattices, with a distribution not leaking information about the trapdoor.

In the GPV framework, `KeyGen` outputs trapdoor pair $(\mathbf{A}, \mathbf{B})$ for a lattice $\mathcal{L}$ with $\mathbf{AB} = 0 \bmod q$, for a public integer q. Note that $\mathcal{L}$ is spanned by $\mathbf{B}$. The signing-verification routine is then:

1. `Sign`$(\mathbf{B}, M)$ first uses a cryptographic hash function `H` to get a vector $\mathbf{m} = \mathtt{H}(M)$ in the ambient space $\mathbb{R}^m$, and computes a preimage $\mathbf{c} = \mathbf{A}^{-1}\mathbf{m}$. Then `Sample`$(\mathbf{B}, \mathbf{c})$ finds a random vector $\mathbf{v} \in \mathcal{L}$ close to $\mathbf{c}$. Thanks to the knowledge of $\mathbf{B}$, the vector $\mathbf{s} = \mathbf{c} - \mathbf{v}$ is indeed short.

2. `Verif`$(\mathbf{A}, M, \mathbf{s}, \gamma)$ computes $\mathbf{m} = \mathtt{H}(M)$, then $\mathbf{c}' = \mathbf{A}\mathbf{s} \bmod q$. If $\mathbf{s}$ is a valid signature, it can be written as $\mathbf{s} = \mathbf{c} - \mathbf{v}$, and then $\mathbf{c}' = \mathbf{A}\mathbf{c} = \mathbf{m} \bmod q$ because $\mathbf{v} \in \mathcal{L}$, so `Verif` first checks this property. But a valid signature is also short, so `Verif` additionally checks that $\|\mathbf{s}\|$ does not exceed a (public, fixed) bound γ. When these two checks are satisfied, the signature is accepted. Otherwise it is rejected.

Of course, the resulting scheme must be *efficient* in practice: consume the *least possible* bandwidth while ensuring *practically fast* signing and verification timings. This implies that:

- `KeyGen` should give *compact* trapdoor pairs (that is, small bit representation);
- `Sample` is fast, and outputs short vectors; the shortness of the output depends on the *quality* of the trapdoor.
- `Sign` outputs the smallest (in bitsize) possible signature from these random vectors.

Lastly, note that intuitively, the shorter the output of `Sample` is, the more difficult it is for an attacker to forge a valid signature, so that only with the knowledge of a very good trapdoor pair should signatures be short vectors. Technicalities start when one realizes that the notion of *good* trapdoor highly depends on how `Sample` is instantiated. Optimizing this aspect is the main guiding principle of our design, and in GPV's instantiation in general.

3.2 Design of `KeyGen`

For the class of NTRU lattices, a trapdoor pair is $(h, \mathbf{B}_{f,g})$, and Prest & Pornin showed that a completion (F, G) can be computed in $O(d \log d)$ time from (f, g). In practice their implementation is as efficient as can be for this technical procedure: it is called `NtruSolve` in Falcon. Their algorithm only depends on the underlying ring and now has a stable version for $\mathbb{Z}[X]/(X^d + 1)$, where $d = 2^n$. We therefore reuse it in our design.

An important concern here is that not all pairs (f, g) and (F, G) provides good trapdoor pairs for `Sample`. Schemes such as Falcon and Mitaka solve this technicality essentially by sieving among all possible bases to find the ones that reach an acceptable quality for the `Sample` procedure. This technique is costly, and many tricks have been used to achieve an acceptable `KeyGen`. We *totally bypass* this sieving routine by redesigning completely how good quality bases can be found — see the

next section for details. This improves the running time of KeyGen, and also increases the security offered by Sample. In any case, the running time of NtruSolve largely dominates the overall time for KeyGen: this is not avoidable because the basis completion algorithm requires the operation of large integers and relatively high-precision floating-point arithmetic.

At the end of the procedure, the secret key contains not only the secret basis, but also the necessary data for Sign and Sample. This additional information can be represented by elements in $K_{\mathbb{R}}$, and is computed during or at the end of NtruSolve. All-in-all, KeyGen outputs:

$$\begin{aligned}\mathtt{sk} &= (\mathbf{b}_1 = (f, g), \mathbf{b}_2 = (F, G), \widetilde{\mathbf{b}}_2 = (\widetilde{F}, \widetilde{G}), \Sigma_1, \Sigma_2, \beta_1, \beta_2),\\ \mathtt{pk} &= (h, q, \sigma_{\mathtt{sig}}, \eta),\end{aligned}$$

where we recall that $h = g/f \bmod q$. These parameters are described more thoroughly in Sections 3.3 and 3.4. A list of their practical value is given in Table 1. Informally, they correspond to the following:

- $(f, g), (F, G)$ is a good basis of the lattice $\mathcal{L}_{\mathrm{NTRU}}$ associated with h, with quality $\mathcal{Q}(f, g) = \alpha$, and $\widetilde{\mathbf{b}}_2$ is the Gram-Schmidt orthogonalization of (F, G) with respect to (f, g);
- $\sigma_{\mathtt{sig}}$ and η are the standard deviation for signature vectors and a tight upper bound on the smoothing parameter of $\mathbb{Z}^d$, respectively;
- $\Sigma_1, \Sigma_2 \in K_{\mathbb{R}}$ represent covariance matrices for two intermediate Gaussian samplings in Sample;
- the vectors β_1 and $\beta_2 \in K_{\mathbb{R}}^2$ represent the orthogonal projections from $K_{\mathbb{R}}^2$ onto $K_{\mathbb{R}} \cdot \mathbf{b}_1$ and $K_{\mathbb{R}} \cdot \mathbf{b}_2$, respectively. In other words, they act as *getCoordinates* for vectors in $K_{\mathbb{R}}^2$. They are used by Sample, and are precomputed for efficiency.

Specification of KeyGen **Algorithm 1** computes the necessary data for signature sampling, and then outputs the key pair. Note that NtruSolve could also compute the sampling data and the public key, but for clarity, the pseudo-code gives these tasks to KeyGen. Figure 2 sketches the key generation procedure.

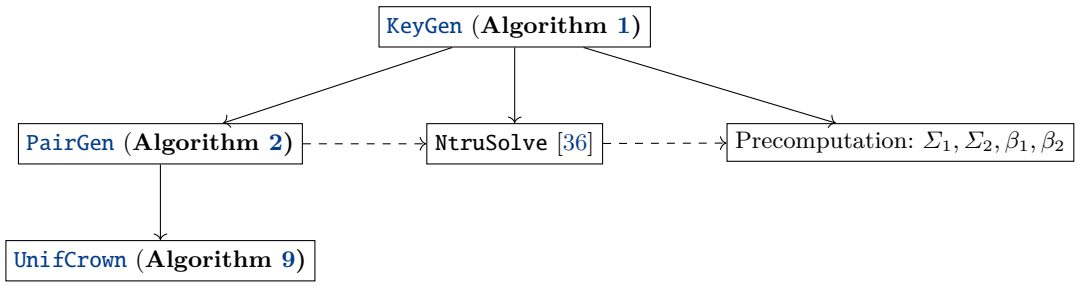


Figure 2: Flowchart of KeyGen

Algorithm 1: KeyGen

Input: A modulus q, a target quality parameter $1 < \alpha$, parameters $\sigma_{\mathtt{sig}}, \eta > 0$

Output: A basis $((f, g), (F, G)) \in R^2$ of an NTRU lattice $\mathcal{L}_{\mathrm{NTRU}}$ with $\mathcal{Q}(f, g) = \alpha$;

```
// Secret basis computation:
```

repeat
  $\mathbf{b}_1 := (f, g) \leftarrow \mathtt{PairGen}(q, \alpha, R_-, R_+)$;
until f *is invertible modulo* q;
$\mathbf{b}_2 := (F, G) \leftarrow \mathtt{NtruSolve}(q, f, g)$;

```
// Public key data computation:
```

$h \leftarrow g/f \bmod q$;
$\gamma \leftarrow 1.1 \cdot \sigma_{\mathtt{sig}} \cdot \sqrt{2d}$; /* tolerance for signature length */

```
// Sampling data computation, in Fourier domain:
```

$\beta_1 \leftarrow \frac{1}{\langle \mathbf{b}_1, \mathbf{b}_1 \rangle_K} \cdot \mathbf{b}_1$;
$\Sigma_1 \leftarrow \sqrt{\frac{\sigma_{\mathtt{sig}}^2}{\langle \mathbf{b}_1, \mathbf{b}_1 \rangle_K} - \eta^2}$;
$\widetilde{\mathbf{b}}_2 := (\widetilde{F}, \widetilde{G}) \leftarrow \mathbf{b}_2 - \langle \beta_1, \mathbf{b}_2 \rangle \cdot \mathbf{b}_1$;
$\beta_2 \leftarrow \frac{1}{\langle \widetilde{\mathbf{b}}_2, \widetilde{\mathbf{b}}_2 \rangle_K} \cdot \widetilde{\mathbf{b}}_2$;
$\Sigma_2 \leftarrow \sqrt{\frac{\sigma_{\mathtt{sig}}^2}{\langle \widetilde{\mathbf{b}}_2, \widetilde{\mathbf{b}}_2 \rangle_K} - \eta^2}$;

$\mathtt{sk} \leftarrow (\mathbf{b}_1, \mathbf{b}_2, \widetilde{\mathbf{b}}_2, \Sigma_1, \Sigma_2, \beta_1, \beta_2)$;
$\mathtt{pk} \leftarrow (q, h, \sigma_{\mathtt{sig}}, \eta, \gamma)$;
return sk, pk;

The technical description on subroutines PairGen (**Algorithm 2**) and NtruSolve are described below.

1. The `PairGen` algorithm generates d complex numbers $(x_j e^{i\theta_j})_{j \le d/2}$ and $(y_j e^{i\theta_j})_{j \le d/2}$ to act as the FFT representations of two *real* polynomial $f^{\mathbb{R}}$ and $g^{\mathbb{R}}$ in $K_{\mathbb{R}}$, respectively. The magnitudes of these complex numbers are sampled in a planar annulus whose small and large radii are set to match a target $\mathcal{Q}(f, g)$ with `UnifCrown` (see also Section 3.6). It then finds close elements $f, g \in R$ by rounding, unless the rounding-error is too large. When the procedure ends, it outputs a pair (f, g) such that $\mathcal{Q}(f, g) = \alpha$, where α depends on the security level.
2. `NtruSolve` is exactly Prest & Pornin's algorithm and implementation [36]. It takes as input $(f, g) \in R^2$ and a modulus q, and outputs $(F, G) \in R^2$ such that $(f, g), (F, G)$ is a basis of $\mathcal{L}_{\text{NTRU}}$ associated with $h = g/f \bmod q$. It does so by solving the Bézout-like equation $fG - gF = q$ in R using recursively the tower of subfields for optimal efficiency.

Specification of `PairGen` This algorithm is a new addition to schemes such as Mitaka. The NTRU module associated with a public key h is morally a 2-dimensional object. When (f, g) and (F, G) are the basis for a given modulus of the lattice, q, we have

$$\det \mathcal{M}_{\text{NTRU}} = (ff^* + gg^*)(\widetilde{F}\widetilde{F}^* + \widetilde{G}\widetilde{G}^*) = q^2 \in R.$$

Similarly, the area of a rectangle is the product of its length and width. This means that all the information about $\widetilde{F}\widetilde{F}^* + \widetilde{G}\widetilde{G}^*$ is determined by (f, g) and q, which explains why *only* (f, g) is needed to know the quality of the basis found by `NtruSolve`. Recall from Section 2 that the quality is

$$\alpha := \mathcal{Q}(f, g) = \max_{1 \le i \le d/2} \max \left(\frac{|\varphi_i(f)|^2 + |\varphi_i(g)|^2}{q}, \frac{q}{|\varphi_i(f)|^2 + |\varphi_i(g)|^2} \right)^{1/2}.$$

Equivalently, a basis has quality α when for all $i \le d/2$, we have

$$\frac{q}{\alpha^2} \le |\varphi_i(f)|^2 + |\varphi_i(g)|^2 \le \alpha^2 q. \tag{1}$$

As this is determined by the evaluations of the polynomials f and g, or equivalently, by their FFT representations, it is natural to sample these polynomials directly via their complex evaluations. This is handled by the subroutine `UnifCrown`, described in Section 3.6 (see **Algorithm 9**).

A technical challenge arises because inverting the `FFT` leads a priori only to real polynomials $f^{\mathbb{R}}$ and $g^{\mathbb{R}} \in K_{\mathbb{R}}$. We thus have to round the coordinates of $f^{\mathbb{R}}$ and $g^{\mathbb{R}}$ to their nearest integers which incurs a small

error. We handle this error by sampling $f^{\mathbb{R}}$ and $g^{\mathbb{R}}$ in FFT-format in a smaller annulus (or crown) with radii

$$R_- = \left(\frac{1}{\alpha} + \delta\right)\sqrt{q} \quad \text{and} \quad R_+ = (\alpha - \delta)\sqrt{q},$$

where δ is a small correction parameter to ensure that the rounding will remain in the correct crown $A(\sqrt{q}/\alpha, \alpha\sqrt{q})$ with a large probability. In practice we take $\delta_{512} = 0.065$ and $\delta_{1024} = 0.3$, values that are set so that ≈ 80 tries on average for $d = 512$, resp. ≈ 5 tries on average for $d = 1024$, of the decoded polynomials (f, g) satisfy Eq.(1).

Let us explain the determination of these values. The decoding errors can be accurately modeled as continuous 2-dimensional Gaussian vectors. This means that in average, the decoding makes an error proportional to the standard deviation of these Gaussians, which are heuristically identically and independently distributed. Due to the good concentration properties of Gaussians, we can fine-tune the tolerance with standard calculations, that match experimental observations. This leads to `PairGen`, where the target quality is by default $\alpha_{512} = 1.17$ and $\alpha_{1024} = 1.64$.

Algorithm 2: `PairGen`

Input: A modulus q, a target quality parameter $1 < \alpha$, two radii parameters $0 < R_- < R_+$

Output: A pair (f, g) with $\mathcal{Q}(f, g) = \alpha$

```
for i = 1 to d/2 do
    x_i, y_i ← UnifCrown(R_-, R_+) ;                    /* see Algorithm 9 */
    θ_x, θ_y ← U(0, 1);
    φ_{f,i} ← |x_i| · e^{2iπθ_x};
    φ_{g,i} ← |y_i| · e^{2iπθ_y};

(f^R, g^R) ← (FFT^{-1}((φ_{f,i})_{i≤d/2}), FFT^{-1}((φ_{g,i})_{i≤d/2}));
(f, g) ← (⌊f_i^R⌉)_{i≤d/2}, (⌊g_i^R⌉)_{i≤d/2};

(φ(f), φ(g)) ← (FFT(f), FFT(g));
for i = 1 to d/2 do
    if q/α^2 > |φ_i(f)|^2 + |φ_i(g)|^2 or α^2 q < |φ_i(f)|^2 + |φ_i(g)|^2 then
        restart;

return (f, g);
```

3.3 Design of Sign

Recall that NTRU lattices live in $\mathbb{R}^{2d}$. Their structure also helps to simplify the preimage computation. Indeed, the signer only needs to compute $\mathbf{m} = \mathtt{H}(M) \in \mathbb{R}^d$, as then $\mathbf{c} = (0, \mathbf{m})$ is a valid preimage: the corresponding polynomials satisfy $(h, 1) \cdot c = m$.

Another interesting feature is that only the *first half* of the signature $(\mathbf{s}_1, \mathbf{s}_2) \in \mathcal{L}_{\text{NTRU}}$ needs to be sent along the message, as long as h is available to the verifier. This comes from the identity $hs_1 = s_2 \bmod q$ defining these lattices, as we will see in the Verif algorithm description. The same identity can also be used to check the validity of signatures only with a hash of the public key h, requiring this time send both $\mathbf{s}_1$ and $\mathbf{s}_2$, but we will not consider this setting here.

Because of their nature as Gaussian integer vectors, signatures can be encoded to reduce the size of their bit-representation. The standard deviation of Sample is large enough so that the $\lfloor \log \sqrt{q} \rfloor$ least significant bits of one coordinate are essentially random. In Falcon, the most significant bits (MSBs) of each coordinate are then sent through an unary (or Huffman) encoding together with their corresponding tails, which already save a nice portion of bandwith. This generic approach is summed-up as the Compress and Decompress functions in Section 3.6.

Following [14], we can go a step further by noticing that instead of encoding each sets of MSB's separately, we can *batch*-encode them as a whole using for example Asymmetric Numeral Systems (ANS). Our signatures then enjoy an additional $7 - 12\%$ compression factor. This will be implemented in a future version of SOLMAE.

In the description above, the scheme cannot output two different signatures for a message. This well-known concern of the GPV framework can be addressed in several ways, for example making a stateful scheme or by hash randomization. Like Falcon, we chose the latter solution for efficiency purposes. In practice, Sign adds a random "salt" $r \in \{0, 1\}^k$, where k is large enough that an unfortunate collision of messages is unlikely to occur, that is, it hashes $(r||M)$ instead of M — our analysis in this regard is identical to that of Falcon. A signature is then $\mathtt{sig} = (r, \mathtt{Compress}(s_1))$. Fig. 3 sketches the signing procedure.

Specification of Sign The signing algorithm Sign handles the hash and the Fourier conversions, leaving the important task of generating the signature vectors to Sample. The hash function is instantiated as explained in Section 3.6. It also generates the salt $r \in \{0, 1\}^k$; the length of the salt

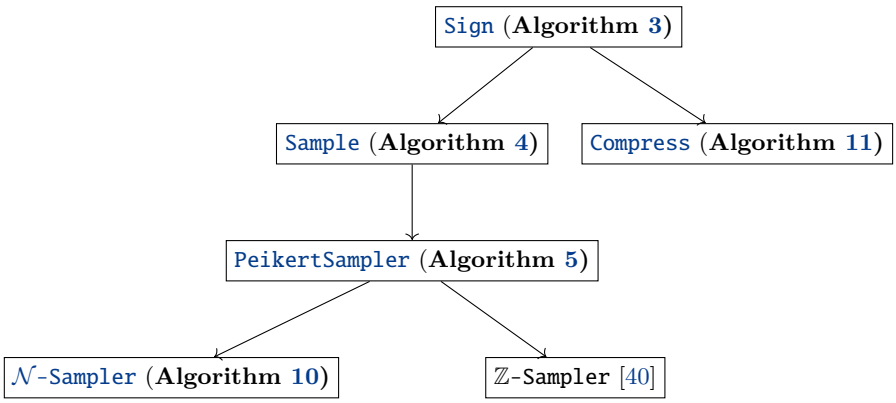


Figure 3: Flowchart of Sign

is obtained by standard conversions from a conservative target bit-security and the maximum number of queries $q_s = 2^{64}$ as $k = 320 = 256 + \log q_s$.

The parameter η is selected as a tight upper bound on *the smoothing parameter* $\eta_\epsilon(\mathbb{Z}^d)$, for some $\epsilon > 0$. Informally, this parameter quantifies the amount of noise to smooth out the discreteness of lattice Gaussian vectors. The value of ϵ is deduced following [38, 10] and the current standard Renyi divergence arguments. The corresponding value for η is then

$$\epsilon = 2^{-41} \quad \text{and} \quad \eta = \frac{1}{\pi}\sqrt{\frac{1}{2} \cdot \log\left(2d\left(1 + \frac{1}{\epsilon}\right)\right)}.$$

Concretely, we have $\eta_{512} \approx 1.338$ and $\eta_{1024} \approx 1.351$. The output is a Gaussian vector $\mathbf{v} \in \mathcal{L}_{\text{NTRU}}$ centered at $\mathbf{c} = (0, \mathtt{H}(r||M))$; the parameter $\sigma_{\mathtt{sig}}$ determines the expected distance from $\mathbf{v}$ to its center, and therefore drives the hardness of the forgery. Its value is determined from [37, 10]:

$$\sigma_{\mathtt{sig}} = \eta \cdot \mathcal{Q}(f, g) \cdot \sqrt{q}.$$

The sampling of the signature vector in **Algorithm 3** is a cascade of different sampling algorithms, Sample, PeikertSampler, and at the deepest level, $\mathbb{Z}$-Sampler. Their specifications follow.

3.4 Design of Sample

Our second main change from Falcon is to rely on the *hybrid sampler* for the Sample procedure. To make sense of this change, we provide the briefest possible outline of Lattice Gaussian sampling — as the sampling

Algorithm 3: `Sign`

Input: A message $M \in \{0,1\}^*$, a tuple
$\mathtt{sk} = ((f,g),(F,G),(\widetilde{F},\widetilde{G}),\sigma_{\mathtt{sig}},\Sigma_1,\Sigma_2,\eta)$, a rejection parameter $\gamma > 0$.
Output: A pair $(r, \mathtt{Compress}(\mathbf{s}_1))$ with $r \in \{0,1\}^{320}$ and $\|(\mathbf{s}_1,\mathbf{s}_2)\| \leq \gamma$.

$r \leftarrow \mathcal{U}(\{0,1\}^{320})$;
$\mathbf{c} \leftarrow (0, \mathtt{H}(r||M))$;
$\hat{\mathbf{c}} \leftarrow \mathtt{FFT}(\mathbf{c})$;
repeat
  $(\hat{s}_1,\hat{s}_2) \leftarrow \hat{\mathbf{c}} - \mathtt{Sample}(\hat{\mathbf{c}},\mathtt{sk})$;
  // $(\mathbf{s}_1,\mathbf{s}_2) \leftarrow D_{\mathcal{L}_{\mathtt{NTRU}},\mathbf{c},\sigma_{\mathtt{sig}}}$
until $\|(\mathtt{FFT}^{-1}(\hat{s}_1),\mathtt{FFT}^{-1}(\hat{s}_2))\|^2 \leq \gamma^2$;
$s_1 \leftarrow \mathtt{FFT}^{-1}(\hat{s}_1)$;
$s \leftarrow \mathtt{Compress}(s_1)$;
return (r,s);

of signature ultimately is the whole core of the entire *fast-GPV* design, this is anyway a required paragraph.

Such algorithms are usually regarded as *randomized decoding procedures*. Babai described two well-known lattice decoding algorithms: the round-off and the nearest-plane. Intuitively speaking, the first just rounds coordinates of the target in the lattice basis to their nearest integer, while the second is more fine-grained and rounds the *Gram-Schmidt* coordinates of the target iteratively, correcting the deviation from the original lattice basis with each decoding step. By randomizing the rounding using Gaussian integers, one obtains either Peikert's sampler [33] or Klein's sampler [28, 16]. The former is more efficient over NTRU lattices, but suffers from a lack of good trapdoors. In other words, signatures are longer with this approach. The latter compensates for its lack of efficiency by large sets of very good trapdoors, and therefore almost optimal signature lengths. As we mentioned, if one accepts a delicate and practically unmaskable implementation involving tree data structures, this drawback can be mitigated.

The hybrid sampler stands in-between, both in efficiency and signature lengths: the Gram-Schmidt orthogonalization and randomized decoding are both done *at the ring level*, so that the algorithm can be seen as a combination of Klein and Peikert's approach. In particular, only two decoding steps are performed for NTRU lattices, compared to $2d$ for the FFO sampler, and the randomization is handled by Peikert's algorithm in the ring, which runs in quasi-linear time: this explains its overall better efficiency. On its first analysis [37], it seemed that good trapdoors for this sampler were costlier to find. Owing to our new `KeyGen` algorithm, this

is no longer a concern, and we are now able to enjoy all the advantages that the hybrid sampler brings versus the FFO sampler: it is simpler to implement, it is more efficient, and it is more friendly to parallelization. Moreover, its masking can be performed with standard, well-understood techniques [10] and online-offline approaches, since the tree structure is now entirely avoided.

Specification of `Sample` `Sample` is an instantiation of the hybrid sampler [37, 10], and can be seen as 2-step randomized decoding, with randomization occurring *at the ring level*, or morally, in a d-dimensional space. In **Algorithm 4** below, note that *all operations are performed in Fourier domain.*

Algorithm 4: `Sample`

Input: A target $\mathbf{c} = (0, \mathbf{c}') \in K_{\mathbb{R}}^2$, a tuple
$\mathtt{sk} = (\mathbf{b}_1 = (f, g), \mathbf{b}_2 = (F, G), \widetilde{\mathbf{b}_2} = (\widetilde{F}, \widetilde{G}), \sigma_{\mathtt{sig}}, \Sigma_1, \Sigma_2, \beta_1, \beta_2)$.

Output: A vector $\mathbf{v} \in \mathcal{L}_{\mathrm{NTRU}}$ with distribution statistically close to $D_{\mathcal{L}_{\mathrm{NTRU}}, \mathbf{c}, \sigma_{\mathtt{sig}}}$.

```
t ← c, v ← 0;
for i = 2 to 1 do
    t_i ← ⟨β_i, t⟩_K;
    z_i ← PeikertSampler(t_i, Σ_i, η) ;        /* z_i ← D_{R, t_i, σ²_sig / ⟨b̃_i, b̃_i⟩} */
    t ← t − z_i b_i, v ← v + z_i b_i;
return v;
```

The randomization is performed by a call to `PeikertSampler` (see **Algorithm 5**), which outputs elements in R with a Gaussian distribution of suitable covariance matrices. In Fourier domain, these covariance matrices are diagonal, and can then be represented by a vector of complex numbers. From Section 3.2, we view $\sigma_{\mathtt{sig}}$ as a constant in $K_{\mathbb{R}}$, and the intermediate standard deviation parameters are

$$\Sigma_i = \sqrt{\frac{\sigma_{\mathtt{sig}}^2}{\langle \widetilde{\mathbf{b}_i}, \widetilde{\mathbf{b}_i} \rangle} - \eta^2} \in K_{\mathbb{R}}.$$

By choice of $\sigma_{\mathtt{sig}}$, they correspond in Fourier domain to positive definite matrices which are actually diagonal with entries the embeddings of Σ_i. The square roots are then easily computed coordinate-wise.

Specification of `PeikertSampler` Again, input, output as well as arithmetic operations are all done in Fourier representation. The principle of `PeikertSampler` is perturbation-based sampling. This translates concretely as a *continuous, elliptic* Gaussian sampling that is easy to handle in Fourier domain with enough precision, combined with a *spherical, discrete* Gaussian sampling of width η, as shown in **Algorithm 5**. The latter is handled by d calls to $\mathbb{Z}$-`Sampler`.

Algorithm 5: `PeikertSampler`

Input: A target $t \in K_{\mathbb{R}}$, parameters $\Sigma, \eta \in K_{\mathbb{R}}^{++}$.
Output: A vector $\mathbf{v} \in R$ with distribution statistically close to $D_{R,t,\sigma}$, where $\sigma = \sqrt{\Sigma^2 + \eta^2}$.

$p \leftarrow \Sigma \cdot \mathcal{N}_1^{K_{\mathbb{R}}}$; /* $p \leftarrow \mathcal{N}_{\Sigma^2}^{K_{\mathbb{R}}}$, done with $\mathcal{N}$-Sampler (Algorithm 10) */
$(p_1, \ldots, p_d) \leftarrow \mathsf{FFT}^{-1}(p)$; /* $(p_i)_i \in \mathbb{R}^d$ */
$(t_1, \ldots, t_d) \leftarrow \mathsf{FFT}^{-1}(t)$; /* $(t_i)_i \in \mathbb{Z}^d$ */
for $i = 1$ *to* d **do**
  $x_i \leftarrow \mathbb{Z}$-`Sampler`$(t_i - p_i, \eta)$;
return $\mathsf{FFT}(x_1, \ldots, x_d)$;

By definition of the parameters, the calls to **Algorithm 5** in **Algorithm 4** output two ring elements with respective covariance $\sigma_{\mathtt{sig}}^2 / \langle \widetilde{\mathbf{b}}_i, \widetilde{\mathbf{b}}_i \rangle_K$, in Fourier representation.

Specification of $\mathbb{Z}$-`Sampler` This step is surprisingly delicate. We reuse the ingenious method of Falcon, and refer to their documentation [40] for the details about the parameters. Below, we give only an informal description of the necessary steps based on [45, 20].

The first concern is the necessity to sample around an arbitrary center $c \in \mathbb{R}$. A technique is to first identify it to its fractional part $\{c\} \in [0, 1)$, and then use a rejection approach to allow a sampling *centered around* 0 instead. This implies computations of the rejection probability, which involve the exponential function and must be performed efficiently at a sufficiently high floating-point precision. Next, the sampling uses a Cumulative Distribution Table (CDT) approach (also known as an inversion-based sampler). The space is saved by using the CDT of a *half* Gaussian distribution combined with a Bernoulli sampler to obtain the sign of the output.

3.5 Design of Verif

The last step of the scheme is thankfully simpler to describe as shown in **Algorithm 6**. Upon receiving a signature (r, s) and message M, the verifier decompresses s to a polynomial s_1 and $\mathbf{c} = (0, \mathtt{H}(r||M))$ to recover the full signature vector $\mathbf{v} = (s_1, s_2)$. If $\mathbf{v}$ is a valid signature, the verification identity is $(h, -1)\cdot(\mathbf{c}-\mathbf{v}) = -\mathtt{H}(r||M) - hs_1 + s_2 \bmod q = 0$, or equivalently the verifier can compute

$$s_2 = \mathtt{H}(r||M) + hs_1 \bmod q.$$

This is computed in the ring R_q, and can be performed very efficiently for a good choice of modulus q using the Number Theoretic Transform (NTT). We currently follow the standard choice (as in Falcon) of $q = 12289$, as the multiplication in NTT format amounts to d integer multiplications in $\mathbb{Z}/q\mathbb{Z}$. In future versions, we will consider different trade-offs between verification efficiency and public-key size. The last step is to check that $\|(\mathbf{s}_1, \mathbf{s}_2)\|^2 \leq \gamma^2$: the signature is only accepted in this case.

The rejection bound γ comes from the expected length of vectors outputted by Sample. Since they are morally Gaussian, they concentrate around their standard deviation; a "slack" parameter $\tau = 1.042$ is tuned to ensure that 90% of the vectors generated by Sample will pass through the loop:

$$\gamma = \tau \cdot \sigma_{\mathtt{sig}} \cdot \sqrt{2d}.$$

Algorithm 6: Verif

Input: A signature (r, s) on M, a public key $\mathtt{pk} = h$, a bound γ.
Output: Accept or reject.

```
s_1 ← Decompress(s);
c ← H(r||M);
s_2 ← c + h s_1 mod q;
if ||(s_1, s_2)||^2 > γ^2 then
    return Reject.
return Accept.
```

3.6 Miscellaneous

In this section we define several supporting algorithms invoked by KeyGen, Sign, and Verif.

Specification of FFT and FFT^{-1}: **Algorithms 7** and **8** show the computation of FFT and its inverse over $K_{\mathbb{R}}$, respectively.

Algorithm 7: FFT

Input: $f \in K_{\mathbb{R}} = \mathbb{R}[X]/(X^d + 1)$.
Output: the FFT representation of f

$\zeta = \exp(i\pi/d)$;
$\varphi(f) \leftarrow (f(\zeta^1), f(\zeta^3), \cdots, f(\zeta^{2d-1}))$;
return $\varphi(f)$

Algorithm 8: FFT^{-1}

Input: $\mathbf{c} = (c_0, \cdots, c_{d-1}) \in \mathbb{C}^d$ such that $c_{d-1-i} = \overline{c_i}$.
Output: $f \in K_{\mathbb{R}}$ such that $\mathbf{c} = \varphi(f)$

$\zeta = \exp(i\pi/d)$;
$\mathbf{V} = \left(\overline{\zeta^{jk}}\right)_{j \in \mathbb{Z}_d, k \in \mathbb{Z}_{2d}^*}$;
$f \leftarrow \frac{1}{d} \cdot \mathbf{V}\mathbf{c}$;
return f

Specification of UnifCrown **Algorithm 9** below outputs a uniformly random element in a fixed planar annulus $A(R_-, R_+) = \{(x, y) \in \mathbb{R}^2 \ : \ R_-^2 \leq x^2 + y^2 \leq R_+^2\}$, from uniformly random numbers in $(0, 1)$ — that is, uniform in the set of floating point numbers with given mantissa and exponent inside $(0, 1)$.

Algorithm 9: UnifCrown

Input: Parameters $0 < R_- < R_+$.
Output: A point (x, y) with uniform distribution in $A(R_-, R_+)$

$u_\rho, u_\theta \leftarrow \mathcal{U}(0, 1)$;
$\rho \leftarrow \sqrt{R_-^2 + u_\rho(R_+^2 - R_-^2)}$;
$x \leftarrow \rho \cdot \cos(\frac{\pi}{2} u_\theta)$;
$y \leftarrow \rho \cdot \sin(\frac{\pi}{2} u_\theta)$;
return (x, y)

To verify this implementation visually, the scatter plot of `UnifCrown` sampler with 10,000 samples was depicted in Figure 4. From this, we can

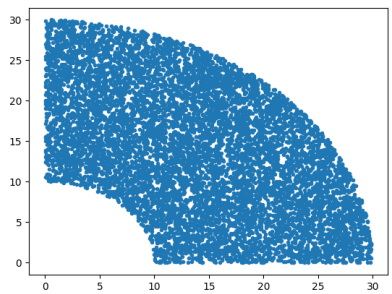


Figure 4: Scatter plot of `UnifCrown` Sampler

see that `UnifCrown` sampler works properly.

Specification of $\mathcal{N}$-`Sampler` In **Algorithm 10**, the first step is the sampling of a continuous Gaussian perturbation with some elliptic covariance E. If Σ is a matrix such that $\Sigma^t \Sigma = E$, then $\mathcal{N}_E = \Sigma \cdot \mathcal{N}_1$.

A multivariate normal distribution is a natural distribution that is used in many fields. The process of generating a sample that follows normal distribution is called Gaussian sampling. In SOLMAE, we use an $\mathcal{N}$-`Sampler` (which is the same as the Gaussian sampler) to generate the FFT representation of d–dimensional standard normal distribution. To generate $\mathbb{R}^d$ vector that follows the multivariate normal distribution of mean $\mathbf{0}$ and variance matrix $\frac{d}{2} \cdot I_d$, we can generate independent $\frac{d}{2}$ many random vectors that follow a bivariate normal distribution with mean $\mathbf{0}$ and variance matrix $\frac{d}{2} \cdot I_2$, then concatenate them together. Also, sampling bivariate normal distribution with mean $\mathbf{0}$ and variance matrix $\frac{d}{2} \cdot I_2$ can be done by using Box–Muller transform [4]. We describe how it works: First, generate u_1 and u_2, two independent random numbers, that follow uniform distribution between 0 and 1. Then, compute R and θ as shown below.

$$R = \sqrt{-d \cdot ln(u_1)}, \quad \Theta = 2\pi \cdot u_2 \tag{2}$$

Finally, calculate X and Y, to convert (R, Θ) into Cartesian coordinates.

$$X = R \cdot \cos(\Theta), \quad Y = R \cdot \sin(\Theta) \tag{3}$$

Algorithm 10: $\mathcal{N}$-`Sampler`

Input: The degree d of R.
Output: Two variables x, y with distribution $\mathcal{N}_{d/2}$

$u_\rho, u_\theta \leftarrow \mathcal{U}(0,1)$;
$\rho \leftarrow \sqrt{-d \ln u_\rho}$;
$x \leftarrow \rho \cdot \cos(2\pi u_\theta)$;
$y \leftarrow \rho \cdot \sin(2\pi u_\theta)$;
return (x, y)

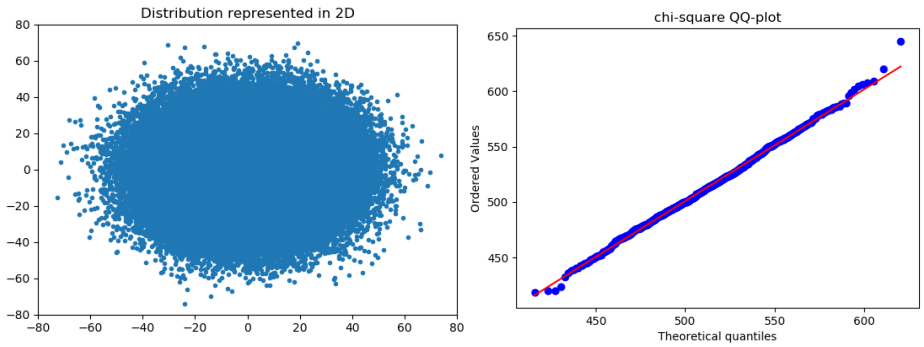
Theorem 1. *(X, Y) in Eq.(3) follows bivariate normal distribution with mean $\mathbf{0}$ and variance matrix $\frac{d}{2} \cdot I_2$.*

Proof. By using the random variable transform theorem stated in [**?**], we show that this theorem holds as follows:

$$\begin{aligned} pdf_{X,Y}(x,y) &= pdf_{U_1,U_2}(u_1,u_2) \cdot \begin{vmatrix} \frac{\partial u_1}{\partial x} & \frac{\partial u_1}{\partial y} \\ \frac{\partial u_2}{\partial x} & \frac{\partial u_2}{\partial y} \end{vmatrix} \\ &= \begin{vmatrix} -\frac{2x}{d} \cdot e^{-\frac{x^2+y^2}{d}} & -\frac{2y}{d} \cdot e^{-\frac{x^2+y^2}{d}} \\ \frac{1}{2\pi} \cdot \frac{-y}{x^2+y^2} & \frac{1}{2\pi} \cdot \frac{x}{x^2+y^2} \end{vmatrix} (u_1 = e^{-\frac{x^2+y^2}{d}}, u_2 = \frac{1}{2\pi} \tan^{-1}(\frac{y}{x})) \\ &= (2\pi)^{-1} \cdot |\Sigma|^{-1/2} exp(-\frac{1}{2}\mathbf{x}^T \Sigma^{-1} \mathbf{x})(\mathbf{x} = (x,y)^T, \Sigma = \frac{d}{2} \cdot I_2) \end{aligned}$$

where *pdf* means the *probability density function*. Thus, we see that (X, Y) in Eq.(3) follows the bivariate normal distribution. □

To check whether the $\mathcal{N}$-`Sampler` used in SOLMAE reference implementation generates the multivariate normal distribution of mean zero and variance matrix $\frac{d}{2} \cdot I_d$ properly, we made a checking program in Python script that produces a sample of size 1,000, then plots the random vectors' projections to $\mathbb{R}^2$ and Chi-square QQ-plot using the built-in library provided in Python. Figure 5 illustrates the 2–dimensional plot of this $\mathcal{N}$-`Sampler`. Figures 5(a) and 5(b) are its scatter plot and QQ–plot, respectively. From this experiment, we can see that this $\mathcal{N}$-`Sampler` works properly.



(a) Scatter plot (b) QQ–plot

Figure 5: Plot of $\mathcal{N}$-`Sampler`

Specification of `Compress` and `Decompress`: We reuse the same method as in FALCON to encode and decode keys and signatures. For the sake of completeness, we describe the compression and decompression functions as depicted in **Algorithms 11** and **12**, respectively. Note that $slen = 8 \cdot |sgn| - 320$ by default where $|sgn|$ denotes the signature size in bytes. Again, we can use the improved encoding technique suggested in [14] to further reduce the signature size which will be implemented in a future version of SOLMAE.

Algorithm 11: `Compress`

Input: A polynomial $s = \sum_{i=0}^{d-1} s_i X^i \in R = \mathbb{Z}[X]/(X^d + 1)$ and an integer $slen$.

Output: A compressed representation of str of s of bitsize $slen$, or $\perp$

$str \leftarrow \{\}$;
for $i = 0$ *to* $d - 1$ **do**
  $str \leftarrow (str \,||\, b)$ where $b = 1$ if $s_i < 0$, $b = 0$ otherwise;
  $str \leftarrow (str \,||\, b_6 b_5 \cdots b_0)$ where $b_j = (|s_i| \gg j)\&0x1$;
  $k \leftarrow |s_i| \gg 7$;
  $str \leftarrow (str \,||\, 0^k 1)$
if $|str| > slen$ **then**
  $str \leftarrow \perp$;
else
  $str \leftarrow (str \,||\, 0^{slen - |str|})$
return str

Algorithm 12: `Decompress`

Input: A bitstring str of bitsize $slen$
Output: A polynomial $s = \sum_{i=0}^{d-1} s_i X^i \in R = \mathbb{Z}[X]/(X^d + 1)$ or $\perp$

```
if |str| ≠ slen then
    return ⊥;
for i = 0 to d − 1 do
    s'_i ← Σ_{j=0}^{6} 2^{6−j} str[1 + j];
    k ← 0;
    while str[8 + k] = 0 do
        k ← k + 1
    s_i ← (−1)^{str[0]} · (s'_i + 2^7 k);
    if s_i = 0 and str[0] = 1 then
        return ⊥
    str ← str[9 + k : ]
if |str| ≠ 0^{|str|} then
    return ⊥;
return s = Σ_{i=0}^{d−1} s_i X^i
```

3.7 Domain parameters

For the sake of clarity, we summarize all the concrete values for the domain parameters as described in Table 1. Note that smoothing η of 1.320 can be used for SOLMAE-512 and SOLMAE-1024 simultaneously for the sake of convenience. In practice, there are no cryptographic differences.

4 Security

4.1 Model for lattice reduction

In all of the following, we follow the so-called *Geometric Series Assumption* (GSA), asserting that a reduced basis sees its Gram-Schmidt vectors' norm decrease with geometric decay. More formally, it can be instantiated as follows for the self-dual BKZ (DBKZ) reduction algorithm of Micciancio and Walter [31]: an output basis $(\mathbf{b}_1, \ldots, \mathbf{b}_n)$ yielded by the DBKZ algorithm with block size β on a lattice $\mathcal{L}$ of rank n satisfies

$$\|\mathbf{b}_i^*\| = \delta_\beta^{d-2(i-1)} \det(\mathcal{L})^{\frac{1}{n}}, \quad \text{where} \quad \delta_\beta = \left(\frac{(\pi\beta)^{\frac{1}{\beta}} \cdot \beta}{2\pi e} \right)^{\frac{1}{2(\beta-1)}},$$

for $\mathbf{b}_i^*$ being the i-th Gram-Schmidt vector of the basis.

Table 1: Domain parameters for SOLMAE family

	SOLMAE-512	SOLMAE-1024
ring degree d	512	1024
dimension $2d$	1024	2048
modulus q	12289	12289
salt length k	320	320
smoothing η	1.320	1.320
smoothness ϵ	2^{-41}	2^{-41}
quality α	1.17	1.64
correction δ	0.065	0.3
lower radius R_-	101.95	100.85
upper radius R_+	122.49	148.54
signature width $\sigma_{\mathtt{sig}}$	173.54	245.62
slack τ	1.04	1.04
rejection bound γ^2	33870790	134150669

4.2 Key recovery attack

The key recovery consists of finding the private secret key (i.e., $f, g \in \mathcal{R}^2$) from the sole data of the public elements q and h. To the best of our knowledge, the most powerful attacks are realized through lattice reduction. It consists of constructing the algebraic lattice over $\mathcal{R}$ spanned by the vectors $(q, 0)$ and $(h, 1)$ (i.e. the public basis of the NTRU key) and retrieve the lattice vector $\mathbf{s} = (\mathbf{g}, \mathbf{f})$ among all possible lattice vectors of the norm bounded by $\|\mathbf{s}\| = \sqrt{2d}\sigma$ (or a functionally equivalent vector, for instance $(\mu g, \mu f)$ for any unit μ of the number field).

We make use of the so-called *projection trick* to avoid enumerating over all the sphere of radius $\sqrt{2d}\sigma$ (which contains around $\left(\frac{2d\sigma^2}{q}\right)^d$ vectors under the Gaussian heuristic). More precisely we proceed as follows. Set β to be the block size parameter of the DBKZ algorithm and start by reducing the public basis with this latter algorithm. Call $[\mathbf{b}_1, \ldots, \mathbf{b}_{2d}]$ the resulting vectors. If we can recover the *projection* of the secret key onto $\mathcal{P}$, the orthogonal space to $\mathrm{Span}(\mathbf{b}_1, \ldots, \mathbf{b}_{2d-\beta-1})$, then we can retrieve the full key in polynomial time by *Babai nearest plane* algorithm to lift it to a lattice vector of the desired norm. Hence it suffices to be able to find the projection of the secret key among the shortest vector of the lattice generated by the last β vectors projected onto $\mathcal{P}$. Classically, sieving on this projected lattice will recover all vectors with a norm smaller than $\sqrt{\frac{4}{3}}\ell$, where ℓ is the norm of the $2d - \beta$-th Gram-Schmidt vector $\mathbf{b}^*_{2d-\beta}$ of

the reduced basis. Under the GSA, we have:

$$\ell = \sqrt{q}\delta_\beta^{-2d+2\beta+2} \approx \left(\frac{\beta}{2\pi e}\right)^{1-\frac{d}{\beta}}.$$

Moreover, considering that $\mathbf{s}$ behaves as a random vector of norm $\sqrt{2d}\sigma$, and using the GSA to bound the norm of the Gram-Schmidt vectors $[\mathbf{b}_1^*, \ldots, \mathbf{b}_{2d-\beta}^*]$, that the norm of its projection over $\mathcal{P}$ is roughly

$$\sqrt{\frac{\beta}{2d}}\|\mathbf{s}\| = \beta^{\frac{1}{2}}\sigma.$$

Hence, we will retrieve the projection among the sieved vectors if $\beta^{\frac{1}{2}}\sigma \leq \sqrt{\frac{4}{3}}\ell$, that is if the following condition is fulfilled:

$$\sigma^2 \leq \frac{4q}{3\beta}\delta_\beta^{4(\beta+1-d)} \tag{4}$$

4.3 Signature forgery by reduction to Approx-CVP

As a Hash-and-Sign paradigm signature, forging a signature stems to feed a lattice point $\mathbf{v}$ at a bounded distance from a random space point $\mathbf{x}$. This Approx-CVP problem can be solved using the so-called *Nearest-Cospace* framework developed in [12]. Under the GSA, Theorem 3.3 of [12] states that under the condition: $\|\mathbf{x}-\mathbf{v}\| \leq \left(\delta_\beta^{2d} q^{\frac{1}{2}}\right)$, decoding can be performed in time poly(d) calls to a CVP oracle in dimension β.

As mentioned in [6] a standard optimization of this attack consists of considering only the lattice spanned by a subset of the vectors of the public basis and decoding within this sublattice. The only interesting subset seems to consist of forgetting the $k \leq n$ first vectors. The dimension is of course reduced by k, at the cost of working with a lattice with covolume $q^{\frac{k}{2(2d-k)}}$ larger. Henceforth the global condition of decoding becomes the (slightly more general) inequality $\|\mathbf{x}-\mathbf{v}\| \leq \min_{k\leq d}\left(\delta_\beta^{2d-k} q^{\frac{d}{2d-k}}\right)$ As such, we need to enforce the condtion:

$$\gamma \geq \min_{k\leq d}\left(\delta_\beta^{2d-k} q^{\frac{d}{2d-k}}\right) \tag{5}$$

4.4 On the other attacks on SOLMAE

In this section, we list the other possible types of attacks on the signature, which are nonetheless irrelevant for the set of parameters we are using.

Algebraic attacks As remarked in the design of NTRU-based schemes (such as for instance Falcon or ModFalcon signatures), there exists a rich algebraic structure in the modules over the convolution ring $\mathcal{R}$ used in SOLMAE. However, there is no known way to improve all the algorithms previously mentioned with respect to their general lattice equivalent by more than polynomial factors (see for instance the speedup on lattice reduction of [26]).

Overstretched NTRU-type As observed in [27], when the modulus q is significantly larger than the magnitudes of the NTRU secret key coefficients, the attack on the key based on lattice reduction recovers the secret key better than the results presented above. This so-called "overstretched NTRU" parameter occurs when $q > (2d)^{2.83}$ for binary secrets, implying that, as is the case for Falcon and other NTRU-based NIST candidates, even *very* significant improvements of this attack would still be irrelevant for the security of the scheme.

Hybrid attacks Odlyzko's meet on the middle attack, or more recently the hybrid attack of Howgrave-Graham [22] which combines a meet-in-the-middle algorithm with a key recovery by lattice reduction were used effectively against NTRU, mainly due to its design using sparse polynomials. As this is not the case (secrets are dense elements in the ring $\mathcal{R}$), their impact is not sufficient to be a problem on the parameter selection of SOLMAE.

4.5 Concrete security

In order to assess the concrete security of our signature scheme, we proceed using the usual cryptanalytic methodology of estimating the complexity of the best attacks against *key recovery attacks* on the one hand, and *signature forgery* on the other.

The analyses translate into concrete bit-security estimates following the methodology of NewHope [1], sometimes called the "core-SVP methodology". In this model [2, 29], the bit complexity of lattice sieving (which is asymptotically the best SVP oracle) is taken as $\lfloor 0.292\beta \rfloor$ in the classical setting and $\lfloor 0.265\beta \rfloor$ in the quantum setting in dimension β.

The resulting security in terms of the sampling quality α is given in Fig. 6 in the dimensions 512 and 1024.

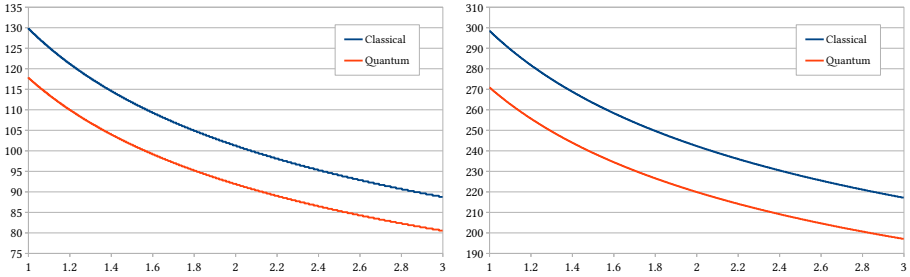


Figure 6: Security (classical and quantum) against forgery as a function of the quality $1 \leq \alpha \leq 3$ of the lattice sampler (left: dimension 512 and right: dimension 1024).

Table 2: Security level for SOLMAE (C: Classical, Q: Quantum)

	SOLMAE-512	SOLMAE-1024
Bit security (C/Q)	127/115	256/232
NIST equivalent	NIST-I	NIST-V

4.6 Side-Channel Resilience

As is the case with Falcon, the difficulty of realizing constant-time implementation lies in the use of floating-point arithmetic. A potential approach to address this problem would be to rely on the work of Pornin [34], which has successfully presented constant-time implementation of floating-point arithmetic by either benefiting from dedicated floating-point hardware when available, or otherwise by emulating floating-point with only integer operations. At the core of `PeikertSampler` is Gaussian sampling over the integers ($\mathbb{Z}$-`Sampler`). We recommend implementing this step by following the isochronous sampler of Howe *et al.* [20]. Their version of $\mathbb{Z}$-`Sampler` essentially invokes a base Gaussian sampler that samples an element with a fixed half Gaussian distribution (which can be made constant-time by naively going through all entries in the CDT) and then rejects that element with a certain probability in such a way that the rejection rate leaks no information about the secret.

Unlike Falcon's tree-based sampler, SOLMAE avoids recursive structures, enabling straightforward application of masking techniques that satisfy t-NI (non-interfering) and t-SNI (strong non-interfering) security. In particular, polynomial multiplication and Gaussian sampling can be masked using standard techniques described in Mitaka paper [11] We

believe this property makes SOLMAE particularly suitable for embedded and side-channel resistant implementations.

5 Performance Analysis

Kim [23] analyzed the asymptotic complexities of Falcon and SOLMAE for the `KeyGen`, `Sign`, and `Verif` procedures, showing that all of them have similar asymptotic complexity of $\Theta(d \log d)$. This makes it difficult to evaluate their exact performance based solely on Python implementations.

Using the reference implementations [24] of Falcon and SOLMAE, which are available on GitHub in C and on the SOLMAE website, respectively, we report their average execution times and provide a performance comparison on the same platform. For further insight into the behavior of the Python implementations of Falcon and SOLMAE, we refer the reader to Kim's monograph [24] published by Springer Briefs in Information Security and Cryptography. A broader performance and implementation security analysis of KpqC competition Round 1 candidates was also conducted by Choi *et al.* [5].

5.1 Description of Platform

The data were collected using two cores of an Intel Xeon CPU E5–2640 v3 @ 2.60 GHz, with 4.00 GB RAM on an Ubuntu 20.04 server, compiled with `CFLAGS = -Wall -Wextra -march=native -O3`. All tests were executed 10,000 times, and average values were reported for fair comparison.

5.2 Performance Comparison

For these experiments, input messages of 1,024 bytes were randomly generated for each of the 10,000 trials using independent key pairs. Performance results for Falcon were obtained using the `speed` tool provided in the official implementation. We first compare the performance of Falcon at different security levels—256 (educational level), 512 (NIST Level I), and 1024 (NIST Level V)—as shown in Figure 7. The `Sign` procedure is observed to be more computationally intensive than `KeyGen` and `Verif`.

The average clock cycles and execution times (μs) for `KeyGen`, `Sign`, and `Verif` using SOLMAE-512 and SOLMAE-1024 are reported in Table 3.

The average time in μs of internal common computations such as FFT, point-wise multiplication and Gaussian sampling,*etc.* which are commonly used for Falcon and SOLMAE families shown in Table 4.

Table 3: Performance comparison of SOLMAE and Falcon

		SOLMAE-512	SOLMAE-1024	Falcon–512	Falcon–1024
KeyGen time	Mcycles	58.77	137.76	33.65	98.65
	time (ms)	22.6	52.97	12.94	37.93
pk size	Bytes	896	1,792	896	1,792
Sign time	Kcycles	758.3	1,512.9	1,353.15	2,792.22
	time (μs)	291.63	581.82	520.4	1,073.81
sgn size	Bytes	666	1,375	666	1,280
Verif time	Kcycles	121.3	289.48	144.31	301.52
	time (μs)	46.66	111.34	55.51	115.97

Table 4: Average time (μs) of common computations for Falcon and SOLMAE

Degree	FFT	poly-add	point-mult	FFT-mul	FFT-poly-div	discrete-Gauss	Gaussian-sampling
512	4.62	0.17	0.21	0.21	0.68	68.85	46.06
1024	8.94	0.3	0.4	0.38	1.37	139.07	89.99

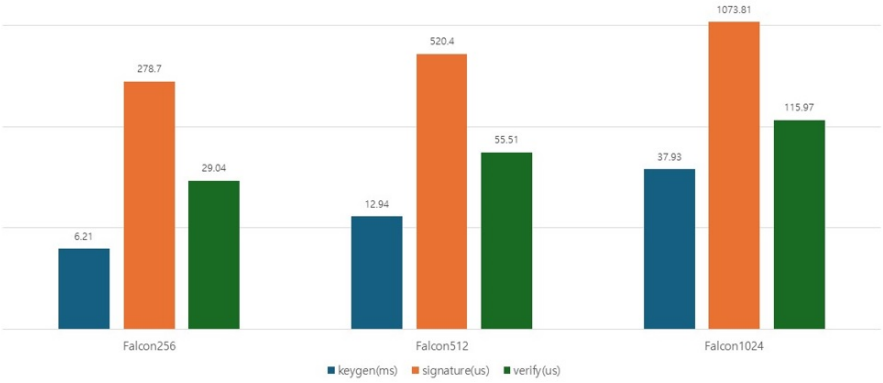


Figure 7: Performance comparison of Falcon-{256, 512, 1024}

From these results, we observe that SOLMAE, without incorporating compression and decompression functions, consistently outperforms Falcon in both `Sign` and `Verif` at the same dimension. However, `KeyGen` is slightly slower due to the additional precomputation required for the private key. The overhead of compression and decompression is negligible.

The average execution times (in μs) of common internal operations—such as FFT, point-wise multiplication, and Gaussian sampling—used in both Falcon and SOLMAE are summarized in Table 4.

Finally, Table 5 compares the performance of SOLMAE-512 with SHAKE-128 and ECDSA P256r1 with SHA-256, using a third-party implementation of our SOLMAE-512 C reference code.

Table 5: Comparison of SOLMAE and ECDSA

		SOLMAE	ECDSA
Specification		512	P256r1
Size(Bytes)	`pk`	1,792	65
	`sgn`	1,375	32
Time	`KeyGen`(ms)	30.21	2.53
	`Sign`(μs)	288.2	2,582.8
	`Verif`(μs)	55.6	7,744.7

Compared to ECDSA, the `KeyGen` procedure of SOLMAE-512 is slower in an online setting.* However, `Sign` and `Verif` are approximately an order of magnitude faster than those of ECDSA P256r1, which is widely used in TLS/SSL. These results demonstrate that the SOLMAE family achieves strong performance without degradation when deployed in quantum-secure PKI and embedded security applications.

6 Coding Rules

We follow the same encoding and decoding formats used in Falcon for the maximum compatibility but some customized headers are used for keypairs and signature of SOLMAE.

Bits and Bytes We restate this part from Falcon specification to increase the readability of this paper. As usual, a byte is a sequence of eight bits (formally, an octet). Within a byte, bits are ordered from left to right. A byte has a numerical value, which is obtained by adding the weighted bits; the leftmost bit, also called *top bit* or *most significant*, has a weight of 128; the next bit has a weight of 64, and so on, until the rightmost bit, which has a weight of 1. Some of the encoding formats defined below use sequences of bits. When a sequence of bits is represented as bytes, the following rules apply:

- The first byte will contain the first eight bits of the sequence; the second byte will contain the next eight bits, and so on. item Within each byte, bits are ordered left-to-right in the same order as they appear in the source bit sequence.
- If the bit sequence length is not a multiple of 8, up to 7 extra padding bits are added at the end of the sequence. The extra padding bits MUST have value zero.

This handling of bits matches widely deployed standards, *e.g.,* bit ordering in the SHA-2 and SHA-3 functions, and BIT STRING values in ASN.1.

Signatures A SOLMAE signature consists of two strings r and s. These strings may conceptually be encoded separately, because the salt r must be known before beginning to hash the message itself, while the s value can be obtained or verified only after the whole message has been processed.

* Note that `KeyGen` in SOLMAE can be executed in a hybrid online/offline manner.

In a format that supports streamed processing of long messages, the salt r would normally be encoded before the message, while the s value would appear after the message bytes. However, we here define an encoding that includes both r and s. The first byte is a header with the following format (bits indicated from most to least significant):

$$1cc0nnnn$$

with these conventions:

- The leftmost bit is 1, and the fourth bit from the left is 0.
- Bits cc are 01 or 10 to specify the encoding method for s. Encoding 01 uses the compression (or decompression) **Algorithm 11** (or **Algorithm 12**); encoding 10 is alternate uncompressed encoding in which each coefficient of s is encoded over a fixed number of bits. The encoding 00 is used for no encoding applied.
- Bits $nnnn$ encodes the hexadecimal value of $0x02$ for SOLMAE-512 and $0x0C$ for SOLMAE-1024 in Table 6.

Public Keys A public key in Falcon and SOLMAE is a polynomial h whose coefficients are considered modulo q. An encoded public key starts with a header byte:

$$1000nnnn$$

with these conventions:

- The four leftmost bits are 1000.
- Bits $nnnn$ encode the same value as the encoding header of signature from Table 6.

Using these conventions, the header byte of encoded public key is $0x82$ and $0x8C$ for SOLMAE-512 and SOLMAE-1024, respectively. After the header byte comes the encoding of h: each value (in the 0 to $q-1$ range) is encoded as a 14-bit sequence (since $q = 12289$, 14 bits per value are used). The encoded values are concatenated into a bit sequence of $14n$ bits, which is then represented as $\lceil 14n/8 \rceil$ bytes.

Private Keys Private keys use the following header byte:

$$1001nnnn$$

with these conventions:

– The four leftmost bits are 1, and the fourth bit from the left is 1.
– Bits $nnnn$ encode the same value as the encoding header of signature and public key from Table 6

Using these conventions, the header byte of encoded private key is $0x92$ and $0x9C$ for SOLMAE-512 and SOLMAE-1024, respectively.

Table 6: Coding value for SOLMAE-d-q

d	q	$nnnn$	Comments
512	12289	0x02	
1024	12289	0x0C	

7 Concluding Remarks

This paper summarizes the important features of SOLMAE which is a lattice-based signature scheme following the hash-and-sign paradigm in the style of GVP framework, and is instantiated over NTRU-lattices. In that sense, it is closely related to, and a successor of, several earlier schemes including Falcon, Mitaka and Antrag. More precisely, SOLMAE offers the unified framework achieving improved efficiency and security trade-offs over Falcon, Mitaka and Antrag.

Falcon has the advantage of providing short public keys and signatures (offering essentially the best bandwidth trade-off among post-quantum constructions) as well as high security levels. In contrast, Mitaka is much simpler to implement, twice as fast in equal dimension, straightforward to parallelize and fully versatile in terms of parameters; however, it has lower security than Falcon in equal dimension, has an even more contrived key generation algorithm that tends to be quite slow, and has somewhat larger keys and signatures at equivalent security levels.

We can conclude that SOLMAE addresses the trade-off among these schemes by offering all their advantages. It uses the same simple, fast, parallelizable signing algorithm as Mitaka, with flexible parameters. However, by leveraging a novel key generation algorithm, SOLMAE achieves the same high security and short key and signature sizes as Falcon and a faster `Sign` procedure. This positions SOLMAE as a highly competitive approach for constructing efficient lattice-based signatures over structured lattices for the embedded systems.

At Real World Post-Quantum Cryptography 2023 (RWPQC2023) symposium held in Tokyo, Prest [39] commented in his talk that *SOLMAE uses the same simple, fast, parallelizable signing algorithm as* Mitaka. *By leveraging a novel key generation algorithm, SOLMAE achieves the same high security and short key and signature size as* Falcon. Moreover, based on SOLMAE specification [25], Cottaar *et al.* [7] concluded that *SOLMAE is close to* Falcon*, but easier to implement* in their evaluation report on KpqC submissions in terms of practical implementation.

Many practical reports on good fit for Falcon were presented on V2V communications [44], TLS certificates [42], DNSSEC [32, 17], low–resource verification of FPGA [3] and Cortex–M3 [18]. It is quite clear that SOLMAE can take all these practices as well due to its fundamental inheritance from Falcon.

Further implementation challenges are left to do next:

- Optimized implementation on various platform.
- Extension of ordinary SOLMAE signature to deterministic [30], message recovery [19] and ring [15] signatures.
- Comparison with the key generation procedure without floating-point arithmetic using Pornin's idea [35] using native FPU instructions with Cortex M7 Raspberry Pi 3 and others [21]

Finally, SOLMAE-512 and SOLMAE-1024 were officially approved as institutional standards [43] by the Telecommunications Technology Association (TTA) of Korea at the end of 2025.

Acknowledgement

The author would like to express sincere gratitude to Mehdi Tibouchi, Alexandre Wallet, Thomas Espitau, Akira Takahashi, and Yang Yu for their voluntary contributions to the preparation of the draft SOLMAE specification.

References

1. Alkim, E., Ducas, L., Pöppelmann, T., Schwabe, P.: Post-quantum key exchange - A new hope. In: Holz, T., Savage, S. (eds.) USENIX Security 2016. pp. 327–343. USENIX Association (Aug 2016) 25
2. Becker, A., Ducas, L., Gama, N., Laarhoven, T.: New directions in nearest neighbor searching with applications to lattice sieving. In: Krauthgamer, R. (ed.) 27th SODA. pp. 10–24. ACM-SIAM (Jan 2016). https://doi.org/10.1137/1.9781611974331.ch2 25

3. Beckwith, L., Kaps, J., Gaj, K.: Fpqa energy consumption of post-quantum cryptography. Fourth PQC Standardization Conference (2022), https://csrc.nist.gov/Events/2022/fourth-pqc-standardization-conference 33
4. Box, G., Mulleri, M.E.: A note on the generation of random normal deviatess. Ann. Math. Statist. pp. 610–611 (1958) 19
5. Choi, Y., Kim, M., Kim, Y., Song, J., Jin, J., Kim, H., Seo, S.C.: KpqBench: Performance and Implementation Security Analysis of KpqC Competition Round 1 Candidates. IEEE Access **12**, 18607–18626 (Feb 2024) 27
6. Chuengsatiansup, C., Prest, T., Stehlé, D., Wallet, A., Xagawa, K.: ModFalcon: Compact signatures based on module-NTRU lattices. In: Sun, H.M., Shieh, S.P., Gu, G., Ateniese, G. (eds.) ASIACCS 20. pp. 853–866. ACM Press (Oct 2020). https://doi.org/10.1145/3320269.3384758 24
7. Cottaar, J., Hövelmanns, K., Hülsing, A., Tanja Lange, M.M., Pellegrini, A., Ravagnani, A., Schäge, S., Trimoska, M., de Weger, B.: Report on evaluation of kpqc candidates. Eindhoven University of Technology (OCt, 31 2023), https://groups.google.com/g/kpqc-bulletin/c/yrfBll_hr14 33
8. Ducas, L., Lyubashevsky, V., Prest, T.: Efficient identity-based encryption over NTRU lattices. In: Sarkar, P., Iwata, T. (eds.) ASIACRYPT 2014, Part II. LNCS, vol. 8874, pp. 22–41. Springer, Heidelberg (Dec 2014). https://doi.org/10.1007/978-3-662-45608-8_2 2
9. Ducas, L., Prest, T.: Fast fourier orthogonalization. In: Proceedings of the ACM on International Symposium on Symbolic and Algebraic Computation, ISSAC 2016, Waterloo, ON, Canada, July 19-22, 2016. pp. 191–198. ACM (2016) 2
10. Espitau, T., Fouque, P.A., Gérard, F., Rossi, M., Takahashi, A., Tibouchi, M., Wallet, A., Yu, Y.: Mitaka: A simpler, parallelizable, maskable variant of falcon. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part III. LNCS, vol. 13277, pp. 222–253. Springer, Heidelberg (May / Jun 2022). https://doi.org/10.1007/978-3-031-07082-2_9 2, 3, 13, 15
11. Espitau, T., Fouque, P.A., Gérard, F., Rossi, M., Takahashi, A., Tibouchi, M., Wallet, A., Yu, Y.: Mitaka: A simpler, parallelizable, maskable variant of falcon. In: Advances in Cryptology – EUROCRYPT 2022. Lecture Notes in Computer Science, vol. 13275, pp. 3–33. Springer (2022). https://doi.org/10.1007/978-3-031-07085-3_1 26
12. Espitau, T., Kirchner, P.: The nearest-colattice algorithm: Time-approximation tradeoff for approx-cvp. Open Book Series **4**(1), 251–266 (2020) 24
13. Espitau, T., Nguyen, T.T.Q., Sun, C., Tibouchi, M., Wallet, A.: Antrag: Annular ntru trapdoor generation. Proc. of Asiacrypt2023, Part VII, Guangzhou, China pp. 3–32 (2023) 2, 3
14. Espitau, T., Tibouchi, M., Wallet, A., Yu, Y.: Shorter hash-and-sign lattice-based signatures. Proc. of CRYPTO 2022, Part II pp. 245–276 (2022) 12, 21
15. Gajland, P., Janneck, J., Kiltz, E.: Ring Signatures for Deniable AKEM: Gandalf's Fellowship. Cryptology ePrint Archive, Paper 2024/890 (2024), https://eprint.iacr.org/2024/890 33
16. Gentry, C., Peikert, C., Vaikuntanathan, V.: Trapdoors for hard lattices and new cryptographic constructions. In: Ladner, R.E., Dwork, C. (eds.) 40th ACM STOC. pp. 197–206. ACM Press (May 2008). https://doi.org/10.1145/1374376.1374407 2, 14
17. Goertzen, J., Stebila, D.: Post-quantum signatures in dnssec via request-based fragmentation. In: Johansson, T., Smith-Tone, D. (eds.) Post-Quantum Cryptography. pp. 535–564. Springer Nature Switzerland, Cham (2023) 33

18. Gonzalez, R., Hülsing, A., Kannwischer, M.J., Krämer, J., Lange, T., Stöttinger, M., Waitz, E., Wiggers, T., Yang, B.Y.: Verifying post-quantum signatures in 8 kb of ram. In: Cheon, J.H., Tillich, J.P. (eds.) Post-Quantum Cryptography. pp. 215–233. Springer International Publishing, Cham (2021) 33
19. Günther, F., Lyubashevsky, V., Schmidt, R.: FALCON with Message Recovery, a Specification. Cryptology ePrint Archive, Paper 2022/1533 (2022), https://eprint.iacr.org/2022/1533 33
20. Howe, J., Prest, T., Ricosset, T., Rossi, M.: Isochronous gaussian sampling: From inception to implementation. In: Ding, J., Tillich, J.P. (eds.) Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020. pp. 53–71. Springer, Heidelberg (2020). https://doi.org/10.1007/978-3-030-44223-1_5 16, 26
21. Howe, J., Westerbaan, B.: Benchmarking and analysing nist pqc lattice-based signature scheme standards on the arm cortex–m7. Fourth PQC Standardization Conference (2022), https://csrc.nist.gov/Events/2022/fourth-pqc-standardization-conference 33
22. Howgrave-Graham, N.: A hybrid lattice-reduction and meet-in-the-middle attack against NTRU. In: Menezes, A. (ed.) CRYPTO 2007. LNCS, vol. 4622, pp. 150–169. Springer, Heidelberg (Aug 2007). https://doi.org/10.1007/978-3-540-74143-5_9 25
23. Kim, K.: Theoretical and empirical analysis of falcon and solmae using their python implementation **14562**, 235–260 (2024) 27
24. Kim, K., Kim, Y.: Asymptotic complexity and performance comparison of falcon and solmae from their C implementation. In: Susilo, W., Deng, R.H., Pieprzyk, J., Wang, H. (eds.) Information Security in a Connected World – Celebrating the Life and Work of Ed Dawson, Lecture Notes in Computer Science, vol. 15600, pp. 192–213. Springer (2024) 27
25. Kim, K., Tibouchi, M., Espitau, T., Takashima, A., Wallet, A., Yu, Y., Guilley, S., Kim, S.: Solmae. KpqC Round 1 Submission (2023), https://kpqc.or.kr/competition.html 33
26. Kirchner, P., Espitau, T., Fouque, P.A.: Fast reduction of algebraic lattices over cyclotomic fields. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part II. LNCS, vol. 12171, pp. 155–185. Springer, Heidelberg (Aug 2020). https://doi.org/10.1007/978-3-030-56880-1_6 25
27. Kirchner, P., Fouque, P.A.: Revisiting lattice attacks on overstretched NTRU parameters. In: Coron, J.S., Nielsen, J.B. (eds.) EUROCRYPT 2017, Part I. LNCS, vol. 10210, pp. 3–26. Springer, Heidelberg (Apr / May 2017). https://doi.org/10.1007/978-3-319-56620-7_1 25
28. Klein, P.N.: Finding the closest lattice vector when it's unusually close. In: Proceedings of the Eleventh Annual ACM-SIAM Symposium on Discrete Algorithms, January 9-11, 2000, San Francisco, CA, USA. pp. 937–941. ACM/SIAM (2000) 14
29. Laarhoven, T.: Search problems in cryptography: from fingerprinting to lattice sieving. Ph.D. thesis, Mathematics and Computer Science (Feb 2016), proefschrift 25
30. Lazar, D., Peikert, C.: Deterministic Falcon Signatures. Cryptology ePrint Archive, Paper 2021/1447 (2021), https://eprint.iacr.org/2021/1447 33
31. Micciancio, D., Walter, M.: Gaussian sampling over the integers: Efficient, generic, constant-time. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017, Part II. LNCS, vol. 10402, pp. 455–485. Springer, Heidelberg (Aug 2017). https://doi.org/10.1007/978-3-319-63715-0_16 22

32. Müller, M., de Jong, J., van Heesch, M., Overeinder, B., van Rijswijk-Deij, R.: Retrofitting post-quantum cryptography in internet protocols: A case study of dnssec (Oct 2020) 33
33. Peikert, C.: An efficient and parallel Gaussian sampler for lattices. In: Rabin, T. (ed.) CRYPTO 2010. LNCS, vol. 6223, pp. 80–97. Springer, Heidelberg (Aug 2010). https://doi.org/10.1007/978-3-642-14623-7_5 14
34. Pornin, T.: New efficient, constant-time implementations of falcon. Cryptology ePrint Archive, Paper 2019/893 (2019), https://eprint.iacr.org/2019/893 26
35. Pornin, T.: Improved key pair generation for falcon, bat and hawk. Cryptology ePrint Archive, Paper 2023/290 (2023), https://eprint.iacr.org/2023/290 33
36. Pornin, T., Prest, T.: More efficient algorithms for the NTRU key generation using the field norm. In: Lin, D., Sako, K. (eds.) PKC 2019, Part II. LNCS, vol. 11443, pp. 504–533. Springer, Heidelberg (Apr 2019). https://doi.org/10.1007/978-3-030-17259-6_17 9, 10
37. Prest, T.: Gaussian Sampling in Lattice-Based Cryptography. Ph.D. thesis, École Normale Supérieure, Paris, France (2015) 3, 13, 14, 15
38. Prest, T.: Sharper bounds in lattice-based cryptography using the Rényi divergence. In: Takagi, T., Peyrin, T. (eds.) ASIACRYPT 2017, Part I. LNCS, vol. 10624, pp. 347–374. Springer, Heidelberg (Dec 2017). https://doi.org/10.1007/978-3-319-70694-8_13 13
39. Prest, T.: Lessons learned talks of the selected candidates: Falcon. Proc. of RWPQC2023, Tokyo (Mar 26, 2023), https://www.youtube.com/watch?v=J0QpSV2xSvM&t=17347s 33
40. Prest, T., Fouque, P.A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Ricosset, T., Seiler, G., Whyte, W., Zhang, Z.: FALCON. Tech. rep., National Institute of Standards and Technology (2020), available at https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions 13, 16
41. Prest, T., Fouque, P.A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Ricosset, T., Seiler, G., Whyte, W., Zhang, Z.: FALCON. Tech. rep., National Institute of Standards and Technology (2022), available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022 2
42. Sikeridis, D., Kampanakis, P., Devetsikiotis, M.: Post-quantum authentication in tls 1.3: A performance study. In: Proceedings of NDSS Symposium, San Diego, California, USA. The Internet Society (Feb 23–26, 2000) 33
43. Telecommunications Technology Association (TTA): Quantum-safe Digital Signature based on NTRU Lattices – Part II: SOLMAE. Tta standard, Telecommunications Technology Association (TTA), Korea (2025), approved as an official TTA standard, December 2025 33
44. Twardokus, G., Bindel, N., Rahbari, H., McCarthy, S.: When cryptography needs a hand: Practical post-quantum authentication for v2v communications. Cryptology ePrint Archive, Paper 2022/483 (2022), https://eprint.iacr.org/2022/483 33
45. Zhao, R.K., Steinfeld, R., Sakzad, A.: Facct: Fast, compact, and constant-time discrete gaussian sampler over integers. IEEE Transactions on Computers **69**(1), 126–137 (2020). https://doi.org/10.1109/TC.2019.2940949 16